

Cross-Dataset ตาม Protocol เดิมรูปแบบ CICIDS2017 และ CSE-CIC-IDS2018

รันวันที่ 10 สิงหาคม 2569 ผลลัพธ์ 280 แถว จาก 4 combination × 2 split mode × 5 seeds × 7 โมเดล

1. สรุปสั้น

ผลสำคัญที่สุดสามข้อ

- การถ่ายทอดข้ามชุดข้อมูลไม่สมมาตรอย่างรุนแรง เทรนด้วย 2018 แล้วทดสอบกับ 2017 ใกล้เคียงประสิทธิภาพได้ถึง 73.9% ของช่วงที่เป็นไปได้ ส่วนที่สลับกันได้เพียง 7.5%
- วิธีแบ่งข้อมูลไม่ได้เปลี่ยนแค่ตัวเลข แต่เปลี่ยนข้อสรุป ภายใต้การแบ่งแบบสุ่มซึ่งเป็นวิธี ที่ protocol กำหนดทั้งสองทิศดูใกล้เคียงกัน (31.1% กับ 25.8%) ความไม่สมมาตรที่เป็นข้อค้นพบ หลักจะมองไม่เห็นเลย
- เมื่อตัดสองคลาสที่พึ่งเชิงโครงสร้างออก การถ่ายทอดทำงานได้จริง stacking ที่เทรนด้วย 2018 ได้ macro-F1 0.806 บนห้าคลาสที่เหลือเมื่อทดสอบกับ 2017

2. Protocol ที่ใช้

ข้อกำหนด	ค่าที่ใช้
Features	77 shared
Classes	7 shared เทรนด้วย 7 คลาสตั้งแต่ต้น
ขนาดตัวอย่าง	300,000 แถวต่อ dataset รักษา rare class
การแบ่ง	train 60 / calibration 10 / test 30
Seeds	5 (42 ถึง 46)
Split mode	สุ่ม และ ตามลำดับเวลา
Tuning	ไม่ทูน ใช้ค่า default เหมือนกันทั้งสองฝั่ง

ไม่ทูนโดยตั้งใจ ช่องว่างที่กำลังวัดอยู่ระดับ 0.4 ถึง 0.6 ส่วนการทูนขยับโมเดลเหล่านี้ได้ 0.01 ถึง 0.05 การนำ hyperparameter ที่ทูนไว้กับ 2018 มาใช้จะให้ความได้เปรียบกับฝั่งหนึ่ง ซึ่งจะปรากฏออกมาเหมือนเป็นผลของ dataset

preprocessor fit ด้วยชุดที่ใช้เทรนเท่านั้น แล้วนำไปใช้กับ test ทั้งสองฝั่งโดยไม่ปรับอะไร นั่นคือความหมายของการถ่ายทอด

3. ข้อมูลที่เข้าสูการทดลอง

	2017	2018
แถวหลัง clean และสุ่ม	300,000	262,701
ที่มา	สุ่มจาก 2,407,275 แถว	จากตัวอย่าง 300,000 แถว
ตัวบอกลำดับเวลา	_row_index	Timestamp

3.1 กฎ clean ใช้ชุดเดียวกันทั้งสองฝั่ง

แปลงค่าอนันต์เป็นค่าว่าง ทั้งแถวที่มีค่าไม่สมบูรณ์ ทั้งแถวซ้ำโดยเทียบเฉพาะ feature กับ label

การ clean 2017 เป็น no-op สมบูรณ์ ไม่มีแถวใดถูกตัดเลย ซึ่งยืนยันว่ากฎที่ใช้ตรงกับกฎที่ สร้างไฟล์ 2017 ขึ้นมาจริง ไม่ใช่แค่ใกล้เคียง

ฝั่ง 2018 ถูกตัดไป 12.4% แยกเป็นค่าไม่สมบูรณ์ 1,738 แถว และแถวซ้ำ 35,561 แถว จำนวนแถวซ้ำสูงกว่าที่ประเมินไว้ตอนแรกมาก เพราะเมื่อไม่นับคอลัมน์ IP, port และ timestamp แล้ว flow จำนวนมากมีค่าเหมือนกันทุกประการ

3.2 จำนวนต่อคลาสไม่เท่ากัน และเป็นข้อมูลสำคัญในการอ่านผล

คลาส	2017	2018	ต่างกัน
Web Attack	2,143	17	126 เท่า
Infiltration	36	2,946	82 เท่า
Brute Force	9,150	1,798	5.1 เท่า
DoS	23,203	8,382	2.8 เท่า
Bot	1,948	3,187	1.6 เท่า
DDoS	15,332	20,654	1.3 เท่า
Benign	248,188	225,717	1.1 เท่า

ชุดคลาสเหมือนกันแล้ว แต่จำนวนไม่เท่ากัน ความต่างของสัดส่วนคือ prior shift ซึ่งเป็นองค์ประกอบ หนึ่งในของ domain shift ที่กำลังวัดอยู่ จึงไม่ควรบังคับให้เท่ากัน แต่ต้องรายงานไว้

สองคลาสที่ต่างกันเกิน 80 เท่า คือสองคลาสที่ได้ F1 เท่ากับศูนย์ในทุกทิศทุกโมเดล

4. ผลที่หนึ่ง การแบ่งข้อมูลแบบสุ่มทำให้เพดานเฟ้อ และเฟ้อไม่เท่ากัน

คะแนนภายในชุดข้อมูลเดียวกัน เทียบระหว่างสองวิธีแบ่ง

โมเดล	2017 ตามเวลา	2017 สุ่ม	เฟ้อขึ้น
mlp	0.5652	0.8815	0.3163
random_forest	0.6652	0.9566	0.2914
catboost	0.7143	0.9695	0.2551
lightgbm	0.7325	0.9775	0.2450
xgboost	0.7314	0.9725	0.2410
stacking	0.7329	0.9730	0.2401
logistic_regression	0.4050	0.6127	0.2077

โมเดล	2018 ตามเวลา	2018 สุ่ม	เฟ้อขึ้น
logistic_regression	0.6810	0.7143	0.0333
mlp	0.7047	0.7344	0.0298
lightgbm	0.7739	0.8019	0.0280
random_forest	0.7146	0.7420	0.0275
catboost	0.7150	0.7423	0.0274
xgboost	0.7719	0.7918	0.0199

โมเดล	2018 ตามเวลา	2018 สุ่ม	เฟลชีน
stacking	0.7039	0.7103	0.0064

2017 เฟลชีน 0.21 ถึง 0.32 ส่วน 2018 เฟลชีนเพียง 0.006 ถึง 0.033 ต่างกันประมาณสิบเท่า

ภายใต้การแบ่งแบบสุ่ม 2017 ได้คะแนนสูงถึง 0.96 ถึง 0.98 ซึ่งเป็นระดับที่ควรตั้งข้อสงสัยทันที พอเปลี่ยนเป็นแบ่งตามลำดับเวลาเหลือ 0.65 ถึง 0.73

เรื่องนี้กระทบการวัดโดยตรง เพราะช่องว่างของการถ่ายทอดวัดเทียบกับเพดานนี้ ถ้าเพดานเฟลชีน ช่องว่างจะดูใหญ่เกินจริง และเมื่อเฟลชีนไม่เท่ากันสองฝั่ง การเปรียบเทียบสองทิศทางก็บิดเบือนไปด้วย

5. ผลที่สอง การถ่ายทอดไม่สมมาตร และวิธีแบ่งข้อมูลปิดบังข้อนี้ไว้

เปอร์เซ็นต์ที่กู้คืนได้ วัดจากช่วงระหว่างการเดาว่าทุกแถวเป็น Benign กับเพดานของชุดนั้น

วิธีแบ่ง	ทิศทาง	สูงสุด	ค่ากลาง
ตามลำดับเวลา	2017 ไป 2018	7.5%	3.6%
ตามลำดับเวลา	2018 ไป 2017	73.9%	51.2%
สุ่ม	2017 ไป 2018	31.1%	9.9%
สุ่ม	2018 ไป 2017	25.8%	15.0%

ภายใต้การแบ่งแบบสุ่ม ทั้งสองทิศดูพอ ๆ กัน ภายใต้การแบ่งตามลำดับเวลา ทิศหนึ่งดีกว่าอีกทิศ เกือบสิบเท่า

ข้อนี้สำคัญกว่าตัวเลขใด ๆ ในรายงาน เพราะมันแปลว่าการเลือกวิธีแบ่งข้อมูลไม่ได้เปลี่ยนแค่ความแม่นยำของการวัด แต่เปลี่ยนข้อสรุปที่จะได้ ถ้ารู้เฉพาะการแบ่งแบบสุ่มตาม protocol จะสรุปว่าการถ่ายทอดทั้งสองทิศยากพอๆ กัน ซึ่งไม่ตรงกับความจริง

5.1 รายละเอียดทิศ 2018 ไป 2017 แบบแบ่งตามลำดับเวลา

โมเดล	เพดาน	ที่ได้อ	กู้คืนได้	ค่าเบี่ยงเบน
stacking	0.7329	0.5756	73.9%	0.0352
lightgbm	0.7325	0.5006	61.5%	0.0204
logistic_regression	0.4050	0.2762	53.3%	0.0000
xgboost	0.7314	0.4376	51.2%	0.0255
mlp	0.5652	0.3049	40.3%	0.0349
catboost	0.7143	0.2830	26.3%	0.0075
random_forest	0.6652	0.1615	6.0%	0.0046

ค่าเบี่ยงเบนข้าม 5 seeds อยู่ระหว่าง 0.000 ถึง 0.035 ซึ่งเล็กกว่าความต่างระหว่างโมเดลและเล็กกว่าช่องว่างที่วัดมาก ผลจึงมีเสถียรภาพพอจะสรุปได้

6. ผลที่สาม ห้าในเจ็ดคลาสถ่ายทอดได้จริง

macro-F1 เฉพาะห้าคลาสที่ไม่ได้พึ่งเชิงโครงสร้าง วัดตอนข้ามชุดข้อมูลแบบแบ่งตามลำดับเวลา

เทรนด้วย	โมเดล	Benign	DDoS	DoS	Brute Force	Bot	รวม 5 คลาส
2018	stacking	0.951	0.620	0.640	0.883	0.936	0.806

เทรนด้วย	โมเดล	Benign	DDoS	DoS	Brute Force	Bot	รวม 5 คลาส
2018	lightgbm	0.942	0.032	0.698	0.889	0.943	0.701
2018	xgboost	0.878	0.029	0.486	0.902	0.765	0.612
2018	mlp	0.933	0.001	0.274	0.914	0.000	0.424
2018	catboost	0.926	0.333	0.674	0.005	0.000	0.388
2018	logistic_regression	0.936	0.053	0.208	0.737	0.000	0.387
2017	lightgbm	0.926	0.000	0.153	0.000	0.181	0.252
2017	ที่เหลือทุกตัว	~0.925	0.000	~0.14	0.000	0.000	~0.21

0.806 คือระดับที่ใช้งานได้จริง ไม่ใช่ 0.24 ที่เห็นเมื่อเฉลี่ยรวมทั้งเจ็ดคลาส

Brute Force ทำได้ถึง 0.914 และ Bot ทำได้ 0.943 ซึ่งเป็นการยืนยันว่าความรู้เรื่องการโจมตีบางประเภทถ่ายทอดข้ามชุดข้อมูลได้จริง ไม่ใช่แค่ความรู้เรื่อง traffic ปกติอย่างที่การทดลอง ก่อนหน้านี้ชี้ไว้

เปรียบเทียบกับ การทดลองก่อนหน้า เมื่อใช้โมเดลที่เทรนด้วย 9 คลาสแล้วยุบผลเป็น 7 ที่หลัง Brute Force ได้เพียง 0.27 พอเทรนด้วย 7 คลาสตั้งแต่ต้นได้ 0.88 ถึง 0.91 การทำให้ชุดคลาสตรงกันจึงมีผลมากกว่าที่ประมาณไว้ก่อนหน้านี้มาก

7. สองคลาสที่ไม่ถ่ายทอดเลย

Infiltration และ Web Attack ได้ F1 เท่ากับศูนย์ในทุกทิศ ทุกโมเดล ทุก seed

Web Attack เป็นปัญหาเชิงจำนวนล้วน ๆ ปี 2018 มีเพียง 17 แฉทั้งหมด เหลือราวสิบแฉตอนเทรนและห้าแฉตอนทดสอบ ไม่ว่าจะเทรนหรือทดสอบก็ไม่มีข้อมูลพอ

Infiltration ซับซ้อนกว่านั้น ทิศ 2018 ไป 2017 มีข้อมูลเทรนราว 1,700 แฉซึ่งเพียงพอ แต่ยังไม่ได้ศูนย์ และแม้แต่ตอนเทรนกับทดสอบภายในชุดเดียวกัน คลาสนี้ก็แค่เพียง 0.05 ถึง 0.13 ในทุกโมเดล ตั้งแต่ logistic regression จนถึง catboost จึงเป็นปัญหาคุณภาพของ label ไม่ใช่ปัญหาที่แก้ได้ ด้วยการปรับวิธีการ

การนำสองคลาสนี้ไปเฉลี่ยรวมทำให้ค่า 0.806 กลายเป็น 0.576 โดยไม่ได้ให้ข้อมูลเพิ่ม จึงควรรายงานแยกพร้อมเหตุผล ไม่ใช่ตัดทิ้งเงียบ ๆ และไม่ไขว่คว้าในค่าเฉลี่ย

8. ข้อสังเกตเรื่องโมเดล

stacking เป็นตัวเดียวที่จับ DDoS ได้ตอนข้ามชุดข้อมูล (0.620 ขณะที่โมเดลเดียวได้ 0.03) และเป็นตัวที่กู้คืนได้สูงสุด การรวมโมเดลจึงช่วยลดการยึดติดกับลักษณะเฉพาะของชุดข้อมูลได้จริง

random_forest ให้ผลตรงข้าม กู้คืนได้เพียง 6.0% ต่ำที่สุดในทุกโมเดล ทั้งที่ภายในชุดข้อมูล เดียวกันทำได้ 0.665 ซึ่งไม่ได้แย่ อาการนี้บ่งชี้ว่ามันเรียนรู้ลักษณะที่ผูกกับชุดข้อมูลมากกว่า ตัวอื่น

9. ข้อจำกัด

1. ปี 2018 ใช้ตัวอย่าง 300,000 แฉที่สุ่มมาก่อนหน้านี้แล้ว ไม่ได้สุ่มใหม่จาก corpus เดิม ขณะที่ 2017 สุ่มจาก 2.4 ล้านแฉ สองปีจึงมาถึง 300,000 แฉด้วยเส้นทางต่างกัน
2. หลัง clean แล้วปี 2018 เหลือ 262,701 แฉ ไม่ครบ 300,000 ตามที่ protocol กำหนด
3. ตัวบ่งชี้อายุต่างกัน 2017 ใช้ลำดับแฉในไฟล์ ส่วน 2018 ใช้ timestamp จริงทั้งคู่เป็นลำดับเวลาเหมือนกัน แต่ได้มาคนละวิธี

4. ช่อง calibration 10% ถูกกันไว้แล้วแต่ยังไม่ได้ใช้ ผลในรายงานนี้ไม่ได้ผ่านการ calibrate
5. ไม่มีการทูน hyperparameter
6. ยังไม่ได้ทำ adversarial validation จึงยังไม่ทราบว่า feature ใดที่บ่งบอกว่า record มาจาก dataset ไหน

10. ขั้นตอนถัดไปที่ข้อมูลชี้ให้เห็น

1. ทำ adversarial validation เป็นขั้นเดียวในแผนเดิมที่ยังไม่ได้ทำ และตอบคำถามว่าช่องว่างที่เหลื่อมมาจาก feature ที่บ่งบอก dataset เท่าไหร่
2. ตรวจสอบว่าทำไม stacking ถึงจับ DDoS ได้ตัวเดียว
ถ้าเข้าใจกลไกนี้อาจนำไปสู่วิธีที่ทำให้โมเดลอื่นทำได้ด้วย
3. ตรวจสอบว่าทำไม random_forest ถ่ายทอดแย่ที่สุด ทั้งที่ภายในชุดข้อมูลทำได้ดี
น่าจะเป็นตัวอย่างที่ขัดที่สุดของการเรียนรู้ลักษณะเฉพาะของชุดข้อมูล
4. พิจารณาการเทรนด้วยสองชุดรวมกัน ถ้าเป้าหมายเปลี่ยนจากการวัดการถ่ายทอด
เป็นการสร้างโมเดลที่ใช้งานได้จริง

11. ไฟล์ผลลัพธ์

อยู่ที่ results/crossdataset/protocol_v1/

- results.csv ผลดิบ 280 แถว
- summary_leakage.csv ตารางหัวข้อ 4
- summary_gap.csv ตารางหัวข้อ 5
- summary_per_class.csv ตารางหัวข้อ 6
- corpora.json รายละเอียดข้อมูลนำเข้าและผลการ clean
- run.log บันทึกการรันทั้งหมด